

Chapter 4 — RL as weighted behavior cloning

Chapter 3 ended with PPO: a robust on-policy algorithm that improves a policy using data the policy itself just generated. This chapter takes the opposite road. It starts from supervised learning on demonstrations, asks how to turn that into *reinforcement* learning, and arrives at an update so simple it fits on one line: clone the data, but weight each sample by how good its action turned out to be.

That one line — advantage-weighted regression — is the load-bearing idea of this entire book. Not because it is the most powerful algorithm here; it is not. It is load-bearing because it is the form of RL that *survives* when the policy stops being a small Gaussian network and becomes a large generative model. Everything in Part III depends on this chapter, so it is worth going slowly.

Here is the route. We price out why on-policy learning is a bad fit for real hardware (4.1), try the obvious alternative and watch it break (4.2), retreat to plain behavior cloning, which cannot break that way but has a hard ceiling (4.3), and then fix the ceiling without giving up the safety: that fix is AWR (4.4). AWAC is a variation on the theme (4.5), and 4.6 states the property that makes the whole book hang together.

4.1 The robot-data reality

4.1.1 What we want, and the obvious thing to try

We want a policy on a physical arm that gets better with experience. You already know a method that does this and have personally made it work: PPO on your differential-drive rover. So the obvious move is to point it at the SO-101.

Before you do, price it out. PPO is **on-policy**, meaning it can only learn from data generated by the current policy (or something extremely close to it): each batch is collected, used for a handful of gradient epochs, and then discarded, because the importance ratio $\rho_t(\theta)$ only corrects for small policy drift. Suppose you budget a modest run of one million environment steps — small by simulation standards; your rover runs used more. Two rates show up in this chapter and both are real: you log teleoperation through the leader arm at 20 Hz, and you run the policy autonomously at 30 Hz. At the autonomous rate, an episode of 20 seconds is 600 steps, so one million steps is about 1,667 episodes.

Now add the part simulation does not have: the reset. After each episode a human has to re-flatten the cloth, or replace the cube, or untangle a cable. Call it 15 seconds, which is optimistic. Each episode therefore costs $20 + 15 = 35$ seconds of wall clock, and:

$$\frac{1,000,000}{600} \approx 1,667 \text{ episodes,} \quad 1,667 \times 35 \text{ s} = 58,345 \text{ s,} \quad \frac{58,345}{3600} \approx 16.2 \text{ hours.}$$

Sixteen hours for one run. Five seeds, the minimum anyone would accept in a paper, is 81 hours — two working weeks of a person standing next to a moving arm, and that is before anything on the arm wears out.

4.1.2 How it fails, and the fix

The deeper problem is not the hours. It is that on-policy learning *throws the hours away*. After the update, that motion is formally worthless for the next update: the policy has moved, so the data is off-distribution, and PPO's clipping machinery exists precisely to stop you from using it too hard. You are renting your data, not owning it.

Meanwhile, look at what is sitting on your disk. Your rl-rover PPO runs saved rollouts at every checkpoint. Your hilserl replay buffers hold hours of SO-101 experience: teleoperation demonstrations, Xbox-pad interventions (in your first working run, 98% of frames were interventions), and autonomous rollouts from a dozen policy versions. None of it came from your current policy. All of it is real physics, real cameras, real outcomes.

Learning about a policy π_θ from data generated by some *other* policy is called **off-policy** learning. Learning entirely from a fixed logged dataset, with no environment interaction at all during training, is **offline RL**. The policy that

actually generated the data — whatever it was — is the **behavior policy**, written π_β . Demonstrations are just the special case where π_β is a competent human holding the leader arm.

Why should this work at all? Because nothing about the data's origin makes its content less true. When your old checkpoint commanded a gripper close at a particular wrist angle and the cloth slipped, the slipping was real; physics does not care which network emitted the command. The hard part — and it is genuinely hard — is extracting that information without fooling yourself.

WORDS YOU WILL HEAR

Off-policy. Learning about one policy from data generated by another. **Offline RL** (older papers: *batch RL*). Training on a fixed dataset with zero environment interaction. **Behavior policy** π_β . Whatever produced the data: a human, a scripted controller, an old checkpoint, or a mixture. **Distributional shift.** The gap between the state-action distribution in your data and the one your new policy will actually visit; actions outside the data's coverage are called *OOD* (out-of-distribution). **Conservatism / pessimism.** The family of fixes — conservative Q-learning (CQL) and implicit Q-learning (IQL) — that deliberately push value estimates *down* on unseen actions, or refuse to evaluate them at all. **Temperature.** A scalar dividing something before an exponential, controlling how sharply the result concentrates.

4.2 Offline RL's failure mode

4.2.1 The obvious thing to try, again

You have the pile. The obvious move is to run a standard off-policy algorithm on it — a Q-learning-style actor-critic. Train a critic $Q(s, a)$ on the logged transitions by temporal-difference regression, then train the actor to output the action that maximizes the critic. No environment interaction needed. This is the structure of the algorithms in your *hil-serl* setup, minus the part where fresh data keeps arriving. It does not work, and the way it fails explains the design of every method in this chapter.

4.2.2 Exactly how it fails

Make it concrete. Your buffer contains SO-101 wrist-flex commands, and because a human generated them through a leader arm at 20 Hz, the per-step wrist delta is always somewhere in $[-10^\circ, +10^\circ]$. Nothing in the dataset has a delta of $+60^\circ$. Not one frame.

Now ask the critic what $Q(s, +60^\circ)$ is. The critic is a neural network. It will answer. It has no mechanism for saying "I have never seen that." It extrapolates smoothly, and the number it returns is whatever the network's shape happens to produce out there. Suppose the true returns in your task all lie in $[0, 1]$ (say, success probability), but the critic's surface happens to peak at $Q(s, +60^\circ) = 8.5$.

Now run the actor update. The actor's job is to find the action maximizing Q . It is a gradient-based optimizer with one objective: climb. It will find that peak, because the peak is exactly what it is searching for. This is the crucial asymmetry: extrapolation error is random in sign, but the optimizer *specifically hunts for the positive errors*. The policy walks toward $+60^\circ$, the arm slams the wrist over, and nothing in the training loop ever contradicts it, because offline there is no environment to send back the bad news.

It gets worse, because of **bootstrapping** — the fact that a value estimate is trained against a target built out of other value estimates rather than out of observed returns. The TD target for a logged transition is $r_t + \gamma \max_{a'} Q(s_{t+1}, a')$, where a' is any action you could take at the next state s_{t+1} , and that maximum is taken over the same inflated surface. So the fantasy value at $+60^\circ$ gets copied into the target for the *previous* state, and then the state before that. A few sweeps through the buffer and the whole value function has drifted upward, anchored to an action that never existed. Everyone who runs vanilla off-policy RL on static data sees this at least once: the loss curves look fine, the values look great, the robot is nonsense.

IN PLAIN WORDS

The critic takes a state $\mathbf{s}$ and an action $\mathbf{a}$ and returns a number: how much total future reward you expect if you take $\mathbf{a}$ here and then behave as usual. The dataset pins that number down only where data exists. Everywhere else it is a guess dressed up as a measurement. The actor is a search procedure over $\mathbf{a}$. Search plus unpinned guesses equals the search finding the guesses. Offline, no feedback loop closes.

A software analogy: a neural critic is like reading past the end of an array in C. There is no bounds check and no exception — you get a plausible-looking number back and the program keeps running. *Where the analogy breaks down:* an out-of-bounds read returns random garbage, so you usually notice fast. Neural extrapolation returns something smooth and confident, and because the optimizer selects for the highest such values, the errors you chase are systematically the flattering ones. The failure is quieter and better disguised than a memory bug.

4.2.3 The fix, and why it works

Every method that succeeds at offline RL applies the same medicine in a different flavor: **keep the policy close to the data**. Some add an explicit penalty on the divergence between π_θ and π_β . Conservative Q-learning (CQL) trains the critic to actively push values *down* on actions the policy proposes but the data does not contain. Implicit Q-learning (IQL) avoids ever evaluating Q at an action outside the dataset. It works for one reason: the queries you make are then queries the data can answer, so the asymmetry that generated the disaster has nothing left to exploit.

The family we build in this chapter enforces that constraint in the most literal way available. AWR, the method of 4.4, never evaluates the policy at an action of its own choosing at all — it only ever looks at actions physically present in the dataset, because it refuses to leave the supervised-learning template. (Its cousin AWAC, in 4.5, does reintroduce a critic and does query it at self-chosen actions; 4.5 explains what makes that survivable. Read 4.4 first.) The supervised-learning template is where we go next.

4.3 Behavior cloning as the base case**4.3.1 The obvious thing to try, one more time**

Behavior cloning (BC) treats control as ordinary supervised learning: collect observation–action pairs $(\mathbf{o}_i, \mathbf{a}_i)$ from a demonstrator and fit a network to predict the action from the observation. As an objective,

$$\max_{\theta} \sum_i \log \pi_{\theta}(\mathbf{a}_i | \mathbf{o}_i),$$

IN PLAIN WORDS

$\pi_{\theta}(\mathbf{a} | \mathbf{o})$ is the probability density the network, with parameters θ , assigns to action $\mathbf{a}$ after seeing observation $\mathbf{o}$ — on your rig, $\mathbf{o}$ is a pair of camera frames plus the joint positions. $\mathbf{a}_i$ and $\mathbf{o}_i$ are the action and observation recorded on frame i , and the sum runs over every frame you logged. The logarithm turns a product of per-frame probabilities into a sum you can minibatch. Maximizing that sum over θ means: make the recorded actions the ones the network would most likely have produced. And notice what is absent — there is no reward anywhere in this expression.

Or, for a deterministic policy that outputs one action rather than a distribution, minimize a mean-squared error between the predicted and recorded actions. That is the whole method. There is no reward, no value function, no exploration, no critic, no environment.

What BC inherits from supervised learning is exactly what makes deep learning pleasant: it is stable, the loss goes down most of the time, it scales with data and parameters, and it works with *any* architecture you can backpropagate through. Nothing in the recipe cares whether π_{θ} is a two-layer MLP or a three-billion-parameter transformer. Hold on to that sentence; it is the one this chapter is ultimately about.

4.3.2 Exactly how it falls short

Its ceiling is just as easy to state: *BC treats every action in the dataset as equally worth imitating*. It clones the demonstrator's mistakes with exactly the same fidelity as their successes.

Make that concrete. Say you record 40 teleop episodes of a grasp on the SO-101. In 12 of them you fumbled the approach and spent about three seconds scrabbling at the object before recovering. At 20 Hz that is 60 frames each, so those episodes contribute roughly 720 frames of scrabbling to a dataset with perhaps 8,000 frames near the grasp. Maximum likelihood does not know scrabbling was a mistake. It sees that from observations that look like "gripper near object," the recorded behavior is a clean grasp most of the time and a scrabble about 9% of the time, and it dutifully reproduces that mixture. You did not train a policy that grasps; you trained one that grasps with a 9% scrabble rate baked in.

A git analogy: plain BC is like building your release by merging every commit in the history with equal trust, including the ones later reverted. AWR, which we are about to derive, weights each commit by whether the release it shipped in passed its tests. *Where the analogy breaks down*: commits are independent and a merge is binary, but robot actions are sequential and correlated — one bad action changes every state you see afterwards, so "blame" spreads across a whole episode rather than sitting on one line. Advantage estimation (Chapter 3) exists to handle that spreading, and reward shaping (Chapter 11) to sharpen it.

4.3.3 What a fix has to preserve

BC has a second, separate problem: compounding error under covariate shift, where small deviations take the policy to states the demonstrator never visited and errors snowball. That thread is picked up in Chapter 13 (DAgger, HG-DAgger, HIL-SERL). Set it aside for now.

So here is the question that defines the rest of the chapter. **Can we keep the supervised loss — with all of its stability and architecture-independence — but make the data's quality matter?**

4.4 AWR derived in slow motion

This is the longest derivation in Parts I and II, and it is the one everything after it stands on, so we take it at walking pace. Here is the map. There are three moves. First, take the data's own action distribution and *tilt* it toward the actions that turned out well, while keeping it on a leash — a limit on how far from the data it may move. The tilt follows a rule we derive rather than guess. The rule has one knob, the temperature β ; 4.4.3 shows where it comes from. Second, *fit* the network to that tilted distribution. Third, notice that the two moves together collapse into something almost embarrassingly simple: plain behavior cloning with one extra number attached to each logged sample. Figure 4.1 is that map as a picture.

Two promises about how this section is written. First: almost nothing in it is new. Almost every step has a twin in Chapter 3 that you have already survived — the surrogate objective of 3.2.3, the trust region of 3.3.1, the KL recipe of 3.3.2, the importance-sampling line of 3.2.2 — and we name each twin as we reach it. The two genuinely new pieces are the Lagrangian, introduced patiently in 4.4.3, and the idea of fitting a network to a target distribution by KL, which 4.4.4 shows you have in fact been doing since 4.3 without the name. Second: one small problem is carried from 4.4.2 to the end — one state of one grasp on your SO-101, three hundred logged frames, three actions, three advantages — and every formula in the section gets evaluated on it with a calculator. If a formula stops making sense, put the toy's numbers in.

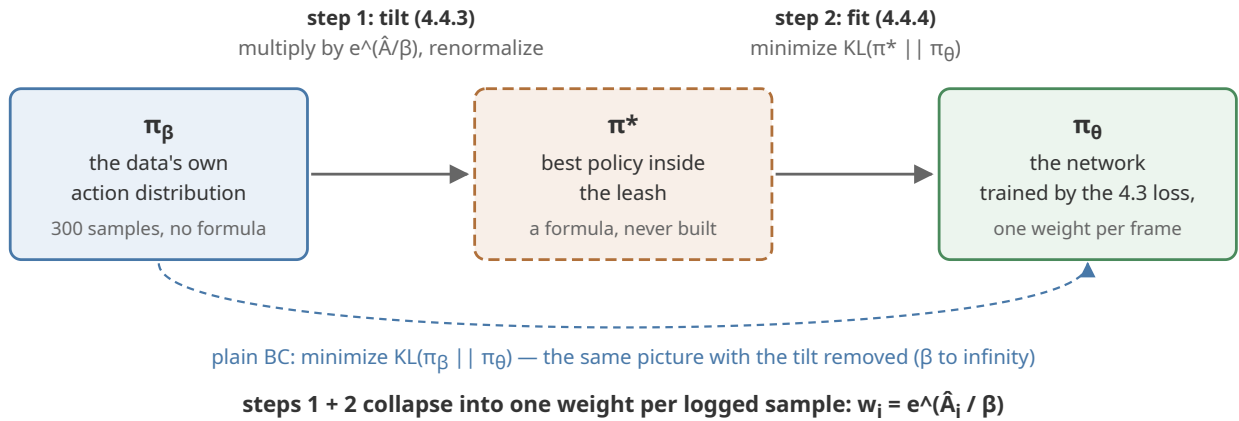


Figure 4.1 — The map of the derivation. Notice that π^* is never actually built; its box is dashed because it exists only as a formula, and step 2 turns "fit the network to π^* " into "fit it to the data you already have, each sample weighted," which is the bold line along the bottom. Plain BC is the same picture with the tilt removed: send $\beta \rightarrow \infty$ and every weight becomes 1.

4.4.1 The obvious thing to try: throw the bad data away

The simplest idea that could work: throw the bad data away. Compute a return for each episode, keep only the best — the top 10% by return, say — and behavior-clone that. This is a real method (filtered BC, or top- k % BC) and a respectable baseline you should run at least once.

Try it on the 40-episode grasp dataset of 4.3.2. Suppose the return penalizes wasted time (a discount factor or a per-step cost both do). Then the twelve scrabbling episodes score lowest. Put the cut just above them and you keep the twenty-eight clean episodes — 70% of the pool here, not the 10% above; the right fraction depends on the pool. The baked-in 9% scramble rate goes to zero. Notice, though, that you could only place that cut because 4.3.2 told you there were exactly twelve bad episodes. On a real pool nothing tells you, and the cut is a guess: the first complaint below. Filtered BC is crude in ways you can feel:

- The threshold is arbitrary. Why 10%? At 5% you have too little data to fit a big network; at 25% you are cloning mediocrity again. Nothing in the problem tells you where to cut.
- It is all-or-nothing. An episode just below the cut contributes nothing, even if it was nearly perfect and failed on one frame. An episode just above it counts in full, even if it succeeded by luck.
- It cannot express degrees. An episode twice as good as another gets the same treatment: included, at the same volume.
- It works at the wrong grain. Quality lives in frames, but the filter operates on episodes. Each of those twelve scrabbling grasps fumbled for three seconds and then recovered: it holds 60 bad frames and a few hundred good ones, and an episode-level cut keeps or discards them together.

What we want is the soft version: every sample stays, but each speaks with a volume proportional to how good it was. Notice the question that sentence hides. Proportional *how*? Linearly, so an action twice as good counts twice? Then what of a below-average action, whose advantage is negative — does it count negatively, pushing the network *away* as in Chapter 2's policy gradient, or a little but positively? There is a whole family of curves that turn "how good" into "how loud," and picking one by taste is a guess.

So we will not guess. We write down what we want as a constrained optimization problem, in the same shape as TRPO's in Section 3.3.1, and solve it; the curve falls out of the solution. To make that solution checkable rather than something to take on trust, the next subsection first sets up a problem small enough to solve on paper.

4.4.2 The constrained problem, on one state of one grasp

State the goal honestly, in words first. We want the policy with the highest expected advantage, subject to not straying far from the data. The second clause is the medicine of 4.2 written into the problem instead of bolted on afterward. Before formalizing either clause, meet the toy.

The SO-101's gripper is 3 cm above a cube and slightly to the left of it. Over many teleop sessions, exactly three different things were ever done from this position:

- $\mathbf{a}_1$: close the gripper now. The fingers close on air beside the cube; the grasp misses.
- $\mathbf{a}_2$: descend 1 cm. The offset is still there; sometimes the grasp lands, sometimes not. Average.
- $\mathbf{a}_3$: shift right 1 cm, then descend. The gripper centers on the cube and the grasp is clean.

Attach an advantage to each: $\hat{A}(\mathbf{a}_1) = -1$, $\hat{A}(\mathbf{a}_2) = 0$, $\hat{A}(\mathbf{a}_3) = +2$. As in Chapter 3, an advantage says how much better than usual an action turned out, judged against what typically happened from this state: $\mathbf{a}_2$ came out level with the baseline $V(\mathbf{s})$ — the value each estimate is measured against — $\mathbf{a}_1$ one unit below it, $\mathbf{a}_3$ two units above. They are estimates, made with Chapter 3's tools, which is why they wear hats.

Now the data. Three hundred logged frames sit at this state, one hundred of each action. So the behavior policy at this state, the distribution the data itself follows, is

$$\pi_{\beta}(\mathbf{a}_1 | \mathbf{s}) = \pi_{\beta}(\mathbf{a}_2 | \mathbf{s}) = \pi_{\beta}(\mathbf{a}_3 | \mathbf{s}) = \frac{1}{3}.$$

Formalize the first clause on the toy. The new policy π is three unknown numbers, $\pi(\mathbf{a}_1), \pi(\mathbf{a}_2), \pi(\mathbf{a}_3)$, non-negative and summing to one. Its expected advantage is

$$\mathbb{E}_{\mathbf{a} \sim \pi}[\hat{A}(\mathbf{a})] = \pi(\mathbf{a}_1) \cdot (-1) + \pi(\mathbf{a}_2) \cdot 0 + \pi(\mathbf{a}_3) \cdot 2.$$

Put the data's own distribution in and you get $-\frac{1}{3} + 0 + \frac{2}{3} = +\frac{1}{3}$: the score of cloning the data exactly as it is, which is what plain BC does. (A true advantage would average to zero over the data's own actions; ours average to $+\frac{1}{3}$ because they are estimates against an imperfect baseline. This does not matter, and 4.4.3 shows why in one sentence.)

The obvious way to push this up is to put all the mass on $\mathbf{a}_3$: $\pi = (0, 0, 1)$ scores **2**, the maximum possible. That is filtered BC in its purest form, and there are two reasons not to trust it. The first is noise: the **+2** is an estimate from one hundred frames, and betting the whole policy on it is betting the run on one number.

The second is worse. The unconstrained problem puts all its mass on whichever action has the largest advantage, and nothing says that action was ever logged. Give the toy a fourth action $\mathbf{a}_4$, the **+60°** wrist flex from 4.2, which no one ever performed. No logged return can score $\mathbf{a}_4$, because it was never taken; the only thing that can put a number on it is a critic asked to extrapolate, which is exactly what happened in 4.2. A critic asked about $\mathbf{a}_4$ will answer, and, as in 4.2, the answer is whatever flattering number its surface happens to hold out there; on the toy's scale call it **+8** — for an action that exists only in the critic's imagination. With nothing but the first clause, the answer is: always do the **+60°** flex. We need the second clause.

Before writing it, notice that the first clause is not new to you. In Section 3.2.3 you maximized the surrogate $L(\theta) = \mathbb{E}_t[\rho_t(\theta) \hat{A}_t]$, and the one-line algebra of 3.2.2 showed that, with ρ the importance ratio $\pi_{\theta}/\pi_{\theta_{old}}$, $\mathbb{E}_{\mathbf{a} \sim \pi_{\theta_{old}}}[\rho \hat{A}] = \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}}[\hat{A}]$: PPO's surrogate is, in words, the expected advantage of the old policy averaged over actions the new policy would choose. Our first clause is that sentence with $\pi_{\theta_{old}}$ renamed π_{β} . It tells the same lie 3.2.3 confessed to — it says nothing about the new policy later visiting different states — and, exactly as in Chapter 3, the lie grows with the distance from the policy that made the data. That is a third reason for a leash — noise and $\mathbf{a}_4$ were the first two — and it is the one Chapter 3 already gave you.

Now the second clause, and again you have seen it. TRPO in Section 3.3.1 took the honest route: since the surrogate is only trustworthy near the policy that generated the data, it constrained the KL divergence to that anchor policy to stay below a small threshold such as 0.01. Do the same thing here with two changes. The anchor is the behavior

policy π_β rather than the previous version of the policy, $\pi_{\theta_{old}}$, because π_β is who made the data. And the KL is written in the other direction — new policy on the left, data on the right; the paragraphs after the box explain why. Here the threshold is written δ :

$$\max_{\pi} \mathbb{E}_{a \sim \pi(\cdot|s)} [A^{\pi_\beta}(s, a)] \quad \text{s.t.} \quad D_{\text{KL}}(\pi(\cdot|s) \parallel \pi_\beta(\cdot|s)) \leq \delta.$$

IN PLAIN WORDS

Read the left half first. π is the new policy we are solving for. $\mathbb{E}_{a \sim \pi(\cdot|s)}[\cdot]$ means "sample an action from the new policy at state s , and average." $A^{\pi_\beta}(s, a)$ is the advantage of the behavior policy: how much better than average taking action a at s was, judged by what the data-generating policy went on to achieve; $\hat{A}$ is our estimate of it. So the left half says: put the mass on above-average actions — on the toy, $-\pi(a_1) + 2\pi(a_3)$. The right half is the leash. $D_{\text{KL}}(\pi \parallel \pi_\beta)$ measures how far the new distribution is from the data's, and δ is how far we allow it: the radius of the leash, in nats, defined below. Small δ : tight leash, timid update. Large δ : loose leash, and the extrapolation disaster of 4.2 returns.

One letter to watch: this δ , with no subscript, is the leash radius, not the TD residual δ_t of Chapter 3.

Read the KL the way Section 3.3.2 taught you: bind the two letters, then ask who is drawing the samples. In Chapter 3 you bound p to the old policy and q to the new one. Here p is the *new* policy π and q is the data's π_β — not TRPO's direction. The a_4 check below shows what this direction buys, and 4.4.3 says in one paragraph what the other direction would have let through. The recipe still works: the new policy draws the action; for each action drawn, compare how likely the new policy was to take it against how often the data took it; average the log of that ratio.

That average is measured in nats: a nat is the unit of a KL whose logarithm is natural, as every **log** in this chapter is. A KL of 1 nat means the log of the ratio averages to 1 over the actions the new policy takes. Undo the log and that says: a typical action is about e , roughly 2.7, times more likely under the new policy than under the data. (Typical in the sense that it is the logarithm that is averaged, so some actions more, some less.) On the toy, with $\pi_\beta = \frac{1}{3}$ everywhere,

$$D_{\text{KL}}(\pi \parallel \pi_\beta) = \sum_a \pi(a) \log \frac{\pi(a)}{\pi_\beta(a)} = \sum_a \pi(a) \log(3\pi(a)).$$

Two edge cases of the KL formula matter here. An action the *new* policy never takes contributes nothing: $0 \cdot \log 0$ read as 0. An action the *data* never took, given any positive probability by the new policy, contributes $+\infty$: $x \cdot \log(x/0)$ with $x > 0$.

Put numbers in. At $\pi = \pi_\beta$ every ratio is 1, every log is 0, and the KL is 0: the leash is slack. At the **greedy** policy — all mass on the single best-scoring action, $\pi = (0, 0, 1)$ — the two actions it never takes contribute 0 and the only surviving term is $1 \cdot \log 3 \approx 1.10$ nats. Now bring back a_4 . The data never took it, so $\pi_\beta(a_4) = 0$. Give it any positive probability and its term is $\pi(a_4) \log(\pi(a_4)/0) = +\infty$: the constraint cannot be satisfied at any δ . This direction of KL reports an infinite leash length for inventing an action the data never took — compare 3.3.2, where in the other direction such a move "barely registers directly." That is what will make AWR structurally safe offline.

One question is left hanging: who chooses δ ? In TRPO you set 0.01 by hand. Here, you will find, nobody does.

4.4.3 Solving it by hand: why the answer is an exponential

Here is the pleasant surprise. TRPO's version of this problem has no closed-form solution — no explicit formula, only an iteration — which is why Section 3.3.3 needed Fisher-vector products, conjugate gradients, and backtracking, and why Chapter 3 ended up with PPO's clip instead. This problem *does* have one. The difference: in TRPO the unknown is θ , millions of network weights, and the KL as a function of them is a curved surface you can only probe locally. Here the unknown is the distribution itself — on the toy, three numbers, with no network in the problem — which is what **nonparametric** means; and the answer is an explicit formula rather than something you iterate toward, which is what **closed form** means. That is why the derivation fits in a few lines. This step is where most readers get lost, so we take it slowly.

Six moves. Price the constraint (the Lagrangian). Differentiate one probability at a time. Solve, and watch the exponential appear. Put the toy in at two temperatures. Try three variants — unequal data, the unlogged action, and a different leash — the last of which is the real answer to "why an exponential." Finally, put numbers on what β buys in leash.

The tool is the **Lagrangian**, and here is all it means. You have a thing to maximize and a rule you must obey. Constrained problems are awkward because the best point may sit on the edge of the allowed region, where derivatives are not zero, so "set the derivative to zero" finds nothing there. Instead of obeying the rule, *charge a price* for breaking it: subtract from the objective a price per unit of leash used, then maximize the resulting unconstrained expression. The price is the **Lagrange multiplier**, and for the right price the priced problem and the constrained one have the same answer (the end of this subsection shows why, once we have numbers to do it with).

An analogy from work: a cloud budget. A hard cap says "this team may use at most δ GPU-hours a month." A price says "each GPU-hour costs the team β dollars from its own budget." Under the price the team optimizes work minus cost, and for the right price it lands on exactly the usage the cap would have allowed; you steer usage by moving the price. *Where the analogy breaks down:* in the accounting version you fix the cap and search for the price that lands on it. AWR does the reverse: you fix the price β , accept whatever leash length comes out, and never search for the δ it corresponds to. The end of this subsection puts numbers on that exchange.

Write it out, on the toy first. Charge a price β per nat of KL and a second price λ per unit by which the probabilities fail to sum to one. (This λ lives only inside this derivation; it is not the GAE λ of Chapter 3.) Call the priced expression $\mathcal{L}$ (a script L, for Lagrangian; not Chapter 3's surrogate $L(\theta)$). With every $\pi_\beta(a) = 1/3$, each $\log(\pi(a)/\pi_\beta(a))$ is $\log(3\pi(a))$, as in the KL formula above:

$$\begin{aligned}\mathcal{L} = & -\pi(a_1) + 2\pi(a_3) \\ & - \beta \left[\pi(a_1) \log(3\pi(a_1)) + \pi(a_2) \log(3\pi(a_2)) + \pi(a_3) \log(3\pi(a_3)) - \delta \right] \\ & - \lambda [\pi(a_1) + \pi(a_2) + \pi(a_3) - 1].\end{aligned}$$

Three lines: the thing we want, minus the leash charge, minus the mass charge. For any list of actions the same expression is

$$\mathcal{L} = \sum_a \pi(a) \hat{A}(a) - \beta \left[\sum_a \pi(a) \log \frac{\pi(a)}{\pi_\beta(a)} - \delta \right] - \lambda \left[\sum_a \pi(a) - 1 \right].$$

For continuous actions the sums become integrals over a and each π is a density; nothing else changes.

Two remarks on what is in the bracket and what is not. The bracket is the constraint written as "leash used minus leash allowed", so it is zero exactly when the leash is taut. And we charge nothing for non-negativity: you will see in a moment that the solution respects it on its own, and the squared-distance contrast later in this subsection shows a leash whose solution does not.

Now "differentiate pointwise," the phrase papers use and rarely explain. It means: take the derivative with respect to *each unknown probability separately*, treating the others as constants, and set each to zero. On the toy that is three derivatives, one per unknown. Start with $\pi(a_3)$, the clean grasp, and go through the three terms of $\mathcal{L}$ that contain it.

The first, $2\pi(a_3)$, has derivative **2**. The last, $-\lambda\pi(a_3)$, has derivative $-\lambda$. The middle one is $-\beta$ times $\pi(a_3) \log(3\pi(a_3))$. Set the $-\beta$ aside for a moment and differentiate what it multiplies, which needs the product rule, the only piece of calculus in the whole derivation: $\frac{d}{dx} x \log x = 1 \cdot \log x + x \cdot \frac{1}{x} = \log x + 1$. Split the term as $\pi(a_3) \log \pi(a_3) + \pi(a_3) \log 3$. The $\log 3$ is a constant, so the second piece has derivative $\log 3$ and the first piece, by the product rule, $\log \pi(a_3) + 1$. The derivative of the whole split term is therefore $\log \pi(a_3) + 1 + \log 3 = \log(3\pi(a_3)) + 1$.

In the general symbols that $\log 3$ is $-\log \frac{1}{3} = -\log \pi_\beta(a_3)$, so the same line reads as follows (the curly ∂ is the partial-derivative sign: the derivative with respect to this one variable, every other variable held fixed — exactly what "pointwise" meant):

$$\frac{\partial}{\partial \pi(a_3)} \left[\pi(a_3) \log \frac{\pi(a_3)}{\pi_\beta(a_3)} \right] = \log \pi(a_3) - \log \pi_\beta(a_3) + 1 = \log \frac{\pi(a_3)}{\pi_\beta(a_3)} + 1.$$

On the toy that is $\log(3\pi(a_3)) + 1$, as we found.

Notice what did *not* appear: δ . It sits in a term with no $\pi(a_3)$ in it, so it vanished the moment we differentiated. Hold that thought. Put the $-\beta$ back in front of the middle piece, assemble the three pieces, and set the sum to zero:

$$2 - \beta \left[\log(3\pi(a_3)) + 1 \right] - \lambda = 0.$$

The equations for a_1 and a_2 are identical with -1 and 0 in place of 2 , and with the same λ , because there is only one mass constraint. Three equations of one shape, so solve the shape once. Write $\hat{A}(a)$ for the advantage and $\pi_\beta(a)$ for the data's share (the 3 above was $1/\pi_\beta(a_3)$), and write plain a for whichever of the three actions we are differentiating with respect to:

$$\hat{A}(a) - \beta \left[\log \frac{\pi(a)}{\pi_\beta(a)} + 1 \right] - \lambda = 0.$$

Now solve for $\pi(a)$, one move per line. Move the bracket across, divide by β , and subtract the 1 that was inside the bracket:

$$\log \frac{\pi(a)}{\pi_\beta(a)} = \frac{\hat{A}(a) - \lambda}{\beta} - 1.$$

Undo the logarithm by exponentiating both sides. This is the moment the exponential is born. The leash's derivative was a logarithm, and the only way to undo a logarithm is to exponentiate. Later in this subsection we swap the leash for a different one and watch a different curve come out. The exponential of a sum is a product of exponentials, so the right side splits into a factor that depends on the action and one that does not; then multiply through by $\pi_\beta(a)$:

$$\pi(a) = \pi_\beta(a) e^{\hat{A}(a)/\beta} e^{-\lambda/\beta - 1}.$$

Look at the last factor. It contains λ and β but not a : it is the same number for every action, and its only job is to make the probabilities sum to one, which is what λ was hired to do. So we never need λ itself: compute the first two factors for every action, add them up, and divide by the total. Call that total Z , so the last factor is simply $1/Z$: $e^{-\lambda/\beta - 1} = 1/Z$, and λ has done its job without ever being computed.

One consequence: add the same constant c to every $\hat{A}$ and every piece $\pi_\beta(a) e^{\hat{A}(a)/\beta}$ is multiplied by the same factor $e^{c/\beta}$, which Z removes. π^* does not care where the advantages' zero sits, which is why the $+\frac{1}{3}$ of 4.4.2 was harmless.

Now put the toy in, before writing anything in general symbols. At $\beta = 1$ the exponential factors are simply $e^{\hat{A}}$. Multiply each by its $\pi_\beta(a) = 1/3$ to get the three unnormalized pieces, add the pieces to get Z , and divide:

```
e^{-1} = 0.3679      e^{0} = 1.0000      e^{2} = 7.3891
pieces pi_beta(a) e^{A(a)}:
(1/3)(0.3679) = 0.1226   (1/3)(1.0000) = 0.3333   (1/3)(7.3891) = 2.4630
Z = (1/3)(0.3679 + 1.0000 + 7.3891) = (1/3)(8.7570) = 2.9190
pi*(a1) = 0.1226 / 2.9190 = 0.0420   -> 4.2%
pi*(a2) = 0.3333 / 2.9190 = 0.1142   -> 11.4%
pi*(a3) = 2.4630 / 2.9190 = 0.8438   -> 84.4%
check: 0.0420 + 0.1142 + 0.8438 = 1.0000
```

The data did the clean grasp one time in three; the best policy inside the leash does it 84 times in a hundred, keeps the average action at 11%, and still performs the miss 4% of the time.

That last number answers the question 4.4.1 left open. A below-average action is not pushed away, the way a negative advantage pushes in Chapter 2's policy gradient; $e^{-1} = 0.37$ is small but positive, so the miss is turned down, never turned off.

The objective rose from 0.3333 to $\mathbb{E}_{\pi^*}[\hat{A}] = 0.0420 \cdot (-1) + 0.1142 \cdot 0 + 0.8438 \cdot 2 = 1.6456$.

Is this π^* really the best point, or merely a place where the derivatives vanish? A stationary point is a point where every derivative is zero. It could in principle be a saddle, a peak along one direction and a valley along another. Here it is not. The expected advantage is a straight line in each probability; the KL is convex in them, because $x \log x$ curves upward, so a sum of such terms has a single valley. A straight line minus a single valley is a single peak. So this stationary point is the maximum, and there is no other.

Now raise β to 5. β divides the advantage before the exponential, which is exactly what 4.1's vocabulary box called a temperature. Raising it warms the tilt. Divide every advantage by 5 *before* exponentiating:

```
e^{-1/5} = e^{-0.2} = 0.8187      e^0 = 1.0000      e^{2/5} = e^{0.4} = 1.4918
sum = 0.8187 + 1.0000 + 1.4918 = 3.3105
Z = (1/3)(3.3105) = 1.1035; dividing (1/3)e^{A/5} by Z is the same as dividing e^{A/5} by 3.3105
pi*(a1) = 0.8187 / 3.3105 = 0.2473    -> 24.7%
pi*(a2) = 1.0000 / 3.3105 = 0.3021    -> 30.2%
pi*(a3) = 1.4918 / 3.3105 = 0.4506    -> 45.1%
E[A] under pi* = -0.2473 + 0 + 2(0.4506) = 0.6539
```

Same data, same advantages, a much gentler tilt: the clean grasp goes to 45% rather than 84%, and the objective to 0.65 rather than 1.65. You will meet these six percentages again in 4.4.5.

Why was one state enough? Peng et al. state the objective with an average over states rather than for a single state. But once the leash is priced with a single β , that price is shared by every state and nothing else joins the states together, so the derivative at any one state looks exactly like the one above. Each state carries its own sum-to-one constraint, so its own λ and its own Z . And a state's frequency in the data multiplies its objective term and its leash term equally, so it cancels from the condition that sets the derivative to zero. The solution therefore has this same shape at every state; their appendix has the derivation in the compressed form you just saw unpacked. Written once in general symbols, with the true advantage A^{π_β} in place of the estimate $\hat{A}$ we used on the toy — the algebra does not care which:

$$\pi^*(a | s) \propto \pi_\beta(a | s) \exp(A^{\pi_\beta}(s, a)/\beta),$$

IN PLAIN WORDS

The best policy inside the leash is the data's own distribution, multiplied pointwise by an exponential factor, then renormalized. π^* is that best policy; the star means "the solution," not "a network." $\pi_\beta(a | s)$ is how often the data took action a at s . $A^{\pi_\beta}(s, a)$ is the behavior policy's advantage from the first key equation; $\hat{A}$ is our estimate of it. $\exp(A^{\pi_\beta}(s, a)/\beta)$ is a positive number: above 1 for an above-average action, below 1 for a below-average one. $\beta > 0$ is the **temperature**; formally it is the Lagrange multiplier of the KL constraint, the price per nat of leash, and in practice you tune it directly and never choose a δ . The $\propto$ means "up to the normalizing constant $Z(s)$ that makes the result sum to one." And if $\pi_\beta(a | s) = 0$, the data never took that action, so $\pi^*(a | s) = 0$ too. This tilt reweights; it never invents.

One notation note for reading papers: the letter used for the data-generating policy varies from paper to paper, and one common choice, π_{old} , is Chapter 3's letter for the previous iterate — which invites exactly the wrong reading. Whatever you see, read it as the policy the samples came from, the role π_β plays here. And Peng et al. write the leash radius as ϵ rather than δ — a leash radius, not PPO's clip parameter. The star in π^* means "the solution"; it is unrelated to the star in $\pi_{0.6}^*$, the Physical Intelligence model of Chapters 8 to 10, where it is part of a product name.

A physical analogy: think of a graphic equalizer on a mixing desk. The recording is fixed; you are only turning up and down the tracks already on it, and the temperature sets how far the faders may travel. *Where the analogy breaks down*: a real fader is bounded, usually to something like a 60 dB range, whereas **exp** is unbounded. One track can be turned up by a factor of three thousand, which is a genuine practical problem and the subject of this chapter's pitfall box.

Equal data for the three actions hides one thing the formula does. Suppose instead the logs were mostly misses: $\pi_\beta = (0.6, 0.3, 0.1)$, the clean grasp only one frame in ten. Keep $\beta = 1$:

```
unnormalized: 0.6 x 0.3679 = 0.2207    0.3 x 1.0000 = 0.3000    0.1 x 7.3891 = 0.7389
Z = 0.2207 + 0.3000 + 0.7389 = 1.2596
pi* = (0.2207, 0.3000, 0.7389) / 1.2596 = (0.1752, 0.2382, 0.5866)
```

The good action was 10% of the data and becomes 59% of π^* ; the miss falls from 60% to 18%. This is what AWR is for: it finds the best behavior latent in your pool, even when it is rare, and makes it the policy's usual behavior.

Now the variant that matters most for safety. Bring back $\mathbf{a}_4$, the $+60^\circ$ wrist flex: never logged, $\pi_\beta(\mathbf{a}_4) = 0$, and carrying the critic's imagined advantage of about $+8$. Then $e^8 \approx 2981$, a factor that would swamp the other three — and $\pi^*(\mathbf{a}_4) = 0 \times 2981 = 0$, with Z and the other three probabilities unchanged. Sit with that, because it is the whole reason this family is safe offline. The **support** of π_β is the set of actions the data gives any probability at all: the actions it ever took. $\mathbf{a}_4$ is outside it. The tilt cannot move mass to an action outside the support, because zero times anything is zero. The failure mode of 4.2 is not being *penalized* here; it is structurally unreachable.

That safety has a price, and you should know it now. Because the tilt only moves mass among actions the data already contains, AWR is a reshuffling operator, not an exploration operator. You just watched it promote a one-in-ten action to the policy's default. It cannot invent a recovery motion nobody ever performed, so improvement is bounded by what the pool holds. That is why the LeHome loop spends most of its engineering on how rollouts are collected and which episodes enter the pool (Chapter 14) rather than on the policy update — and why exploration for this class of policy is still open.

You may still be asking the question this subsection promised to answer: why an *exponential*? What about the problem made it come out that way? The cleanest way to see it is to change the leash and watch the answer change. Suppose "close to the data" were measured by squared distance, $\sum_a (\pi(a) - \pi_\beta(a))^2 \leq \delta$, the first thing an engineer would reach for. The Lagrangian is the same one with the new leash in the bracket:

$$\mathcal{L}' = \sum_a \pi(a) \hat{A}(a) - \beta \left[\sum_a (\pi(a) - \pi_\beta(a))^2 - \delta \right] - \lambda \left[\sum_a \pi(a) - 1 \right].$$

Differentiate pointwise as before: the derivative of $(\pi(a) - \pi_\beta(a))^2$ with respect to $\pi(a)$ is $2(\pi(a) - \pi_\beta(a))$ by the chain rule, so setting the derivative to zero gives $\hat{A}(a) - 2\beta(\pi(a) - \pi_\beta(a)) - \lambda = 0$, and solving,

$$\pi(a) = \pi_\beta(a) + \frac{\hat{A}(a) - \lambda}{2\beta},$$

a *linear* tilt: add a slice proportional to the advantage. It looks harmless. Put the toy in at $\beta = 1$. The probabilities must sum to one: $\sum_a \pi(a) = \sum_a \pi_\beta(a) + \sum_a (\hat{A}(a) - \lambda)/2 = 1 + (\sum_a \hat{A}(a) - 3\lambda)/2$, which must equal 1, so $\lambda = \frac{1}{3} \sum_a \hat{A}(a) = \frac{1}{3}$. Then

```
pi(a1) = 1/3 + (-1 - 1/3)/2 = 1/3 - 2/3 = -1/3
pi(a2) = 1/3 + ( 0 - 1/3)/2 = 1/3 - 1/6 = 1/6
pi(a3) = 1/3 + ( 2 - 1/3)/2 = 1/3 + 5/6 = 7/6
```

A negative probability for the miss and a probability above one for the clean grasp. To rescue this you would have to bolt on a third constraint, $\pi(a) \geq 0$, and clip: an extra rule the KL leash never needs, because an exponential is positive on its own.

Now bring in $\mathbf{a}_4$, with $\pi_\beta(\mathbf{a}_4) = 0$ and the imagined advantage of 8. The linear recipe gives $\pi(\mathbf{a}_4) = 0 + (8 - \lambda)/2$, positive for any λ below 8. With $\mathbf{a}_4$ in the sum, normalization sets $\lambda = (-1 + 0 + 2 + 8)/4 = 9/4$, so $\pi(\mathbf{a}_4) = (8 - 9/4)/2 = 23/8$, about 2.9: a "probability" of nearly three for an action nobody ever performed. The squared-distance leash *invents the $+60^\circ$ action*.

Clipping the negative entries to zero does not save it either: carry the normalization through with the clip in place and the only consistent answer is $\pi = (0, 0, 0, 1)$, everything on the action nobody ever took. A linear tilt has no notion of "the data never did this."

So the leash is not cosmetic. KL is built out of logarithms, so its derivative is a logarithm. Setting that derivative to zero makes $\log(\pi/\pi_\beta)$ linear in the advantage. Undoing a logarithm gives an exponential, which is always positive and multiplies π_β , so it is zero exactly where the data is zero. The exponential is not a modeling choice; it is what "stay close to the data, measured in KL" turns into once you solve it.

Papers call the result an **exponential tilt** of π_β , or a Boltzmann or Gibbs distribution; the vocabulary box in 4.4.4 collects those names.

One last contrast, and then the question of δ . Had we written the leash in TRPO's direction, $D_{\text{KL}}(\pi_\beta \| \pi)$, the a_4 term would be $0 \cdot \log(0/\pi(a_4)) = 0$, whatever $\pi(a_4)$ is. That direction charges for mass drained *from* a_1, a_2, a_3 — 3.3.2's "it registers because that mass had to come from somewhere" — but not one nat for where the mass *lands*. So whatever mass the leash lets leave the logged actions is free to land on the $+60^\circ$ flex, and with an imagined advantage of $+8$ waiting there, that is exactly where the optimizer sends it. (Solve that version of the problem on the toy at $\beta = 1$ and 87% of the policy ends up on a_4 .) The direction 4.4.2 chose reads infinity for the same move. That is why the leash points the way it does.

Finally, who chooses δ ? Nobody, and now you can see the mechanism. δ vanished from the derivative, so the solution only knows the price β . But each price implies a leash length actually used — the KL between π^* and π_β — and you can compute it with the toy's KL formula $\sum_a \pi^*(a) \log(3\pi^*(a))$:

```
# ln = natural log, the same log as in every formula above
beta = 1: 0.0420 ln(0.1260) + 0.1142 ln(0.3426) + 0.8438 ln(2.5314)
          = 0.0420(-2.0715) + 0.1142(-1.0712) + 0.8438(0.9288)
          = -0.0870 - 0.1223 + 0.7837 = 0.574 nats
beta = 5: 0.2473 ln(0.7419) + 0.3021 ln(0.9063) + 0.4506 ln(1.3518)
          = 0.2473(-0.2985) + 0.3021(-0.0984) + 0.4506(0.3014)
          = -0.0738 - 0.0297 + 0.1358 = 0.032 nats
```

Those are two rows of a table you can extend in either direction. Push β up and the tilt fades toward plain BC; push it down and the tilt sharpens toward the greedy policy, whose KL of $\log 3$ you computed in 4.4.2:

beta	pi*(a1)	pi*(a2)	pi*(a3)	KL(pi* pi_beta)	
inf	0.3333	0.3333	0.3333	0.000	(no tilt: plain BC)
100	0.3289	0.3322	0.3389	0.0001	
5	0.2473	0.3021	0.4506	0.032	
1	0.0420	0.1142	0.8438	0.574	
0.25	0.0000	0.0003	0.9997	1.096	
-> 0	0	0	1	1.099 = ln 3	(greedy: the leash cannot be longer)

So on this data, choosing $\beta = 5$ is the same decision as choosing a leash of radius $\delta \approx 0.03$, and $\beta = 1$ is the same as choosing $\delta \approx 0.57$. Why is it the same decision? Call the expected advantage a policy's *plain score* and the Lagrangian its *priced score*. For any distribution the mass term is zero, so the priced score is the plain score minus β times (KL used minus δ). Look at that bracket: a policy that uses less leash than δ is paid a credit, $\beta(\delta - \text{KL})$, on top of its plain score. Every policy inside the leash therefore has a priced score at least as large as its plain score. π^* has the highest priced score of any distribution: it is the single peak we found, a straight line minus a single valley. π^* also uses exactly δ of leash, so its credit is zero and its priced and plain scores are equal. Chain those: $\text{plain}(\pi) \leq \text{priced}(\pi) \leq \text{priced}(\pi^*) = \text{plain}(\pi^*)$. No policy inside the leash beats π^* on the plain score, which is what the constrained problem asked for.

Do not compare these radii with TRPO's 0.01 in 3.3.1: that is an average over states of a continuous Gaussian's KL, paid again at every update, and the scales have nothing to say to each other.

The multiplier is literally an exchange rate. Going from $\beta = 5$ to $\beta = 1$ bought **0.99** more expected advantage ($1.6456 - 0.6539$) for **0.54** more nats of leash ($0.574 - 0.032$): about 1.8 units of advantage per nat, averaged over that stretch.

At any single temperature the rate is exactly β : five units of advantage per nat at $\beta = 5$, one per nat at $\beta = 1$. That is a general fact about Lagrange multipliers, used here without proof. You can check it roughly on the table. Take two neighboring rows, compute the objective $-\pi^*(a_1) + 2\pi^*(a_3)$ for each, and divide its change by the change in KL; the answer always lands between the two rows' β s ($5 \rightarrow 1$ gives the 1.8 above; $100 \rightarrow 5$ gives about 9.5). That is what a Lagrange multiplier *is*: the rate at which the best achievable objective rises per unit of constraint.

Every temperature you set implies a leash radius you could compute the same way — one per state. At the same price a state with spread-out advantages uses more leash than a calm one: the toy's 0.57 nats at $\beta = 1$, against about 0.003 nats for a state whose three equally logged actions scored $\hat{A} = (-0.1, 0, +0.1)$. A single radius quoted for a whole dataset can only be an average of those. Nobody computes any of them, because the price is the more convenient dial: β enters the solution directly, while a chosen δ would have to be converted into its β by a search.

4.4.4 The projection step: from a formula to a network

There is one problem left, and it is not small. π^* is a formula, not a network. It is defined as "whatever π_β is, times an exponential," and we do not have π_β . On the toy we read it off by counting; for a camera image on the SO-101, the probability that the human would have sent this wrist delta given these pixels is not a number anyone possesses. We have samples of it — three hundred at the toy's state, one per logged frame on the real robot. So we need a network π_θ as close to π^* as its architecture allows, trained from samples alone, and able to generalize to states the formula has never seen. Papers call this the **projection** of π^* onto the network's family: the member of that family nearest to π^* . Nearest in what sense is the first decision; turning that choice into something you can compute from logged frames alone is the second. We take it in four steps.

Step one: you already know how to project onto a network, because plain BC is a projection. Section 4.3 trained BC by maximizing $\sum_i \log \pi_\theta(a_i | o_i)$. That objective has a standard name, **maximum likelihood**: pick θ to make the logged actions as probable as possible. Or, dividing by the number of frames N , which changes nothing about where the maximum is: maximize $\frac{1}{N} \sum_i \log \pi_\theta(a_i | o_i)$. Now fix one state, as 4.4.2 and 4.4.3 did — the toy's — and look only at its frames; the sum over states comes back near the end of this subsection. At one state the observation is fixed, so for the next few paragraphs write $\pi_\theta(a)$ for $\pi_\theta(a | o)$. An average over recorded samples estimates an expectation under whatever produced them, and π_β produced them, so BC is maximizing an estimate of $\mathbb{E}_{a \sim \pi_\beta}[\log \pi_\theta(a)]$. Add and subtract $\log \pi_\beta(a)$ inside that expectation, which changes nothing:

$$\mathbb{E}_{a \sim \pi_\beta}[\log \pi_\theta(a)] = \mathbb{E}_{a \sim \pi_\beta} \left[\log \frac{\pi_\theta(a)}{\pi_\beta(a)} \right] + \mathbb{E}_{a \sim \pi_\beta}[\log \pi_\beta(a)] = -D_{\text{KL}}(\pi_\beta \| \pi_\theta) + \mathbb{E}_{a \sim \pi_\beta}[\log \pi_\beta(a)].$$

The first piece is minus a KL, by the 3.3.2 definition with $p = \pi_\beta$ and $q = \pi_\theta$: $\log(\pi_\theta/\pi_\beta)$ is $-\log \pi_\beta + \log \pi_\theta$. The second piece has no θ in it; it is a property of the data alone, and no change to the network moves it. So maximizing BC's objective is the same as minimizing $D_{\text{KL}}(\pi_\beta \| \pi_\theta)$. Behavior cloning is the instruction: make the network's distribution as close as possible, in KL, to the data's distribution.

A KL is never negative and is zero only when the two distributions agree, so $-D_{\text{KL}} + \text{constant}$ peaks exactly at $\pi_\theta = \pi_\beta$. On the toy, BC's objective is $\frac{1}{3}[\log \pi_\theta(a_1) + \log \pi_\theta(a_2) + \log \pi_\theta(a_3)]$, maximized at $\pi_\theta = \pi_\beta$. AWR is the identical instruction with π^* in place of π_β :

$$\min_{\theta} D_{\text{KL}}(\pi^* \| \pi_\theta).$$

Same machinery, one target swapped — the lower and upper paths of Figure 4.1. On the toy, a network that could represent any three-number distribution would land exactly on $\pi^* = (0.042, 0.114, 0.844)$; in general it cannot, and the projection lands on the nearest thing it can.

Step two: the direction of the KL. The arguments have swapped relative to 4.4.2: there the thing we were solving for sat on the left of the KL, here π_θ sits on the right. KL is not symmetric (3.3.2), so this is a choice, not a typo; the 3.3.2 recipe says why. Ask who draws the samples. $D_{\text{KL}}(\pi^* \parallel \pi_\theta)$ averages over actions drawn from π^* , and step four shows that our logged actions can be reweighted into exactly that average. The other direction, $D_{\text{KL}}(\pi_\theta \parallel \pi^*)$, averages over actions drawn from the network. At each such action it needs $\log \pi^*(a) = \log \pi_\beta(a) + \hat{A}(a)/\beta - \log Z$. Two of those pieces we do not have. $\pi_\beta(a)$ is the behavior density at an action the network chose. $\hat{A}(a)$ is an advantage at an action the network chose — a critic query at a self-chosen action, the pattern that sent the wrist to $+60^\circ$ in 4.2.

Two names for what you just saw. Target on the left, model on the right is the **forward KL**, what maximum likelihood minimizes; the other direction is the **reverse KL**. By this naming, the leash of 4.4.2 was a reverse KL: $D_{\text{KL}}(\pi \parallel \pi_\beta)$ has the thing being solved for on the left. The projection here is a forward KL: the target π^* is on the left and the model π_θ on the right.

So the leash and the projection point in opposite directions, and both are right, because they do different jobs. The leash was chosen for what it charges an action the data never took: infinity, the a_4 check of 4.4.2. The projection was chosen for who can draw the samples: the data supplies actions from π_β , and step four reweights them into π^* ; at the network's own draws we would need π_β and $\hat{A}$, which we do not have.

An exam analogy: forward KL grades the network on the questions in the answer key — the logged actions — and we have that key. Reverse KL grades the network on questions it wrote for itself, which needs an answer key for every possible question, including the $+60^\circ$ wrist flex nobody asked. *Where the analogy breaks down:* in a real exam both directions are possible and the choice is about fairness; here the reverse direction is not unfair but uncomputable.

Step three: drop the term without θ . Write the forward KL out in full, as we did for BC:

$$D_{\text{KL}}(\pi^* \parallel \pi_\theta) = \mathbb{E}_{a \sim \pi^*}[\log \pi^*(a \mid s)] - \mathbb{E}_{a \sim \pi^*}[\log \pi_\theta(a \mid s)].$$

Look at the two terms. The first contains no θ at all: it is the negative entropy of π^* (entropy: how spread out a distribution is; a constant here), a fixed number once π^* is fixed, and 4.4.3 fixed it. Nothing you do to the network changes it, so it contributes nothing to the gradient and you drop it. Minimizing the KL therefore reduces to one instruction: maximize $\mathbb{E}_{a \sim \pi^*}[\log \pi_\theta(a \mid s)]$, the expected log-likelihood the network assigns to actions drawn from π^* — the one quantity left to maximize.

Step four: the change of measure, which is the step that makes everything work. We cannot sample from π^* ; we can only sample from π_β , because that is what a dataset is. A **change of measure** rewrites an average under one distribution as an average under another by attaching a ratio of the two to each sample — and you have done one already. It is importance sampling, Section 3.2.2, where $\rho = \pi_\theta/\pi_{\theta_{\text{old}}}$ turned an average under $\pi_{\theta_{\text{old}}}$ into one under π_θ . Do the identical thing with π^* in the role of π_θ and π_β in the role of $\pi_{\theta_{\text{old}}}$, one equality per move:

$$\begin{aligned} \mathbb{E}_{a \sim \pi^*}[\log \pi_\theta(a)] &= \sum_a \pi^*(a) \log \pi_\theta(a) \\ &= \sum_a \pi_\beta(a) \frac{\pi^*(a)}{\pi_\beta(a)} \log \pi_\theta(a) \\ &= \sum_a \pi_\beta(a) \frac{e^{\hat{A}(a)/\beta}}{Z} \log \pi_\theta(a) \\ &= \mathbb{E}_{a \sim \pi_\beta} \left[\frac{e^{\hat{A}(a)/\beta}}{Z} \log \pi_\theta(a) \right]. \end{aligned}$$

Read it top to bottom. The first equality writes the expectation as a sum. The second multiplies and divides by $\pi_\beta(a)$. Do it only over actions the data contains; the other actions contribute nothing to the left side anyway, because π^* is zero there (4.4.3), so leaving them out changes nothing. The third substitutes the closed form $\pi^*(a) = \pi_\beta(a)e^{\hat{A}(a)/\beta}/Z$, and the two $\pi_\beta(a)$ cancel. The fourth reads the result as an expectation under π_β . It is

Section 3.2.2's line with the letters renamed and $f = \log \pi_\theta$. On the toy the first form puts weight **0.0420** on $\log \pi_\theta(a_1)$; the last form puts $\frac{1}{3} \times 0.3679/2.9190 = 0.0420$ on it. Same number, reached from the side we can actually sample.

Now the twist that 3.2.2 could not offer. Look at the ratio here: $\pi^*/\pi_\beta = e^{\hat{A}/\beta}/Z$. The behavior density canceled — the most important sentence in the section. Section 3.2.2 warned that the importance ratio requires the actual density of the action under both policies, and that Chapter 7 is where that stops being satisfiable. Here that requirement has vanished. The *weight* needs no density evaluation of anything — not π_β , not π^* , not the network's own.

$\hat{A}$ you compute before training. Z needs no density either: it is the average of $e^{\hat{A}/\beta}$ over the logged actions at that state, 2.919 on the toy at $\beta = 1$, and a few paragraphs from here it is dropped. The only density left anywhere in the expression sits outside the weight, in the *loss*: the ordinary BC term, $\log \pi_\theta$ at a logged action.

That single cancellation is why AWR needs no model of the behavior policy, and it is why Section 4.6 will be able to replace even $\log \pi_\theta$ with any supervised loss. When 4.6 says AWR never forms an importance ratio, this is what it means: w_i is a ratio only on paper. The densities that would make it one have already canceled, and what remains is a number you read off $\hat{A}_i$.

The last thing to see is that an average under π_β is exactly what a dataset average estimates. On the toy, for any function g of the action, $\sum_a \pi_\beta(a) g(a) = \frac{1}{300} \sum_{i=1}^{300} g(a_i)$, because each action appears in exactly the proportion π_β says. So

$$\mathbb{E}_{a \sim \pi^*} [\log \pi_\theta(a)] = \frac{1}{300} \sum_{i=1}^{300} \frac{e^{\hat{A}_i/\beta}}{Z} \log \pi_\theta(a_i).$$

On the toy this is exact, because the counts are exactly 100 each. On a real dataset the frames are a finite sample of π_β , so the same line holds only approximately, as an estimate — the same status as every sample average in Chapter 3. Either way the right side is a sum over the frames you have, each multiplied by its own number: at $\beta = 1$, **7.3891**/ Z for each of the hundred frames of a_3 , $1/Z$ for each frame of a_2 , **0.3679**/ Z for each frame of a_1 . Add up the weights before dividing by Z :

100 x 0.3679 =	36.79	share =	36.79 / 875.7 =	4.2%
100 x 1.0000 =	100.00	share =	100.00 / 875.7 =	11.4%
100 x 7.3891 =	738.91	share =	738.91 / 875.7 =	84.4%
total	= 875.70			

Those are the three probabilities of π^* from 4.4.3, reappearing as each action's share of the training signal. The projection lands the network on π^* not by building π^* but by letting each logged frame speak with the volume π^* would have given it. For continuous actions the sum over a becomes an integral and every π a density; the dataset average estimates that integral the same way, one weighted term per frame.

One more step, small but worth saying. Everything above was one state. Every other state in the data gets the same weighted average over its own frames, with its own $Z(s)$. Weight each state's average by how often that state appears in the logs and add them up. That weighting is not a choice: the objective is evaluated on the states the data visited — the frozen state distribution 3.2.3 confessed to, which 4.4.2 repeated — and it is the average over states that 4.4.3 mentioned. Each frame is one state-action pair and sits in exactly one per-state average, so the result is a single sum over every logged frame, divided by N . That is how the $\frac{1}{300} \sum$ at one state becomes the $\sum_i$ over the whole dataset in the key equation below; the $1/N$ is one constant for the entire sum, so it does not move the argmax. The $1/Z(s)$ is not one constant, and it is the next loose end.

That loose end first within a state. Written honestly, $\pi^*(a | s) = \pi_\beta(a | s) \exp(\hat{A}/\beta)/Z(s)$, where $Z(s)$ normalizes the distribution at state s . It depends on the state but not on the action, so within a state it scales every sample's weight by the same number: on the toy, dividing all three hundred weights by $Z = 2.919$ leaves the shares 4.2%, 11.4%, 84.4% exactly as before.

Across states Z does vary. Computing it exactly would mean averaging $e^{\hat{A}/\beta}$ over every action at every state. On the toy that average ran over three hundred frames; on a real robot a continuous state is visited once, holds one logged action, and offers no average to take. So nobody computes $Z(s)$, and every implementation drops the $1/Z$. Be clear that this is a simplification, and know which way it leans. A state whose logged advantages spread more widely, or sit higher on average, has a larger Z than a state whose actions all came out near average: at the toy's state $Z = 2.919$ at $\beta = 1$, while the calm state of 4.4.3, whose three equally logged actions scored $\hat{A} = (-0.1, 0, +0.1)$, has $Z = \frac{1}{3}(0.905 + 1.000 + 1.105) = 1.003$. Dropping the $1/Z$ therefore lets the varied state speak roughly three times louder ($2.919/1.003$, about 2.9), relative to the calm one, than the derivation said.

Every implementation accepts that, for two reasons. Standardizing $\hat{A}$ per batch puts all advantages on one scale, which keeps $Z(s)$ within a modest range across states instead of letting one state's Z run away to astronomical values. The variation that remains is a small distortion, tolerated in practice rather than proved harmless. And whatever uniform scale is left is indistinguishable from a change in the learning rate. That is why the weight in every implementation you will read is simply $\exp(\hat{A}_i/\beta)$, with no $1/Z(s)$ in sight.

One last shift, notational and deliberate. Everything in the derivation was written over states s , because that is where the MDP lives and where the advantage is defined; only the BC objective borrowed from 4.3 was already in θ . On a real robot you never see s ; you see an observation $\mathbf{o}$ — camera frames plus joint positions. From here on the policy is conditioned on $\mathbf{o}$, and every statement about s is to be read with $\mathbf{o}$ substituted in. Together, the four steps plus the three housekeeping moves — sum over states, drop $1/Z$, read s as $\mathbf{o}$ — give weighted maximum likelihood on the data you already have. Maximum likelihood is 4.3's objective under the name step one gave it. Weighted means each frame's term is scaled by its own w_i :

$$\theta \leftarrow \arg \max_{\theta} \sum_i w_i \log \pi_{\theta}(a_i | \mathbf{o}_i), \quad w_i = \exp(\hat{A}_i/\beta).$$

IN PLAIN WORDS

This is behavior cloning with one scalar multiplier per sample. θ is the network's weights, and $\arg \max_{\theta}$ says: find the weights that make the sum as large as possible. The sum runs over every logged frame i . $\log \pi_{\theta}(a_i | \mathbf{o}_i)$ is the ordinary BC term: how likely the network thinks the recorded action a_i was, given the recorded observation $\mathbf{o}_i$. w_i is a positive number attached to that sample and nothing else: the ratio π^*/π_{β} with Z absorbed, 0.37, 1.00, or 7.39 on the toy. $\hat{A}_i$ is that sample's estimated advantage, computed before training. β is the temperature: dividing by it before exponentiating sets how far the weights spread. As $\beta \rightarrow \infty$ every weight goes to 1: plain BC. As $\beta \rightarrow 0$ the largest advantage takes over: clone the winner.

This is **advantage-weighted regression** (AWR; Peng et al., 2019, generalizing reward-weighted regression, Peters & Schaal, 2007). Two housekeeping notes. First, the letter β does double duty in the standard notation: it is the temperature here, and it is also the subscript in π_{β} for the behavior policy. They are unrelated; everyone finds this mildly annoying and nobody has renamed them. Second, $\hat{A}_i$ comes from tools you already own from Chapter 3: a Monte Carlo return minus a learned value baseline, $\hat{A}_t = G_t - V(s_t)$, or GAE with parameters γ and λ (GAE's λ , not the Lagrange multiplier of 4.4.3). Nothing new is required.

WORDS YOU WILL HEAR

Lagrangian, Lagrange multiplier. A constrained problem rewritten with a price per unit of constraint used; the multiplier is that price. **Closed form.** An explicit formula for the solution, no iteration needed. **Nonparametric.** A distribution written out directly, not the weights of a network. **Forward vs. reverse KL.** Forward KL, $D_{\text{KL}}(\text{target} \parallel \text{model})$: what maximum likelihood minimizes; the model must cover everything the target does — which is why the single Gaussian of 5.2.3 puts its mean in the mug, between two swings. Reverse KL, $D_{\text{KL}}(\text{model} \parallel \text{target})$: content to sit on one peak and ignore the other. **Normalizing constant, partition function Z .** Whatever you divide by so the probabilities sum to one. **Change of measure, importance weight.** An average under one distribution rewritten as an average under another; the ratio each sample carries is the importance weight. **Exponential tilt, Boltzmann or Gibbs distribution.** A base distribution multiplied by $e^{(\cdot)/\beta}$ and renormalized; β is the temperature. **Effective sample size.** $(\sum_i w_i)^2 / \sum_i w_i^2$: how many equally weighted samples your weighted set is worth.

Figure 4.2 is the whole algorithm as one picture, and Figure 4.3 shows what the temperature actually does to a dataset.

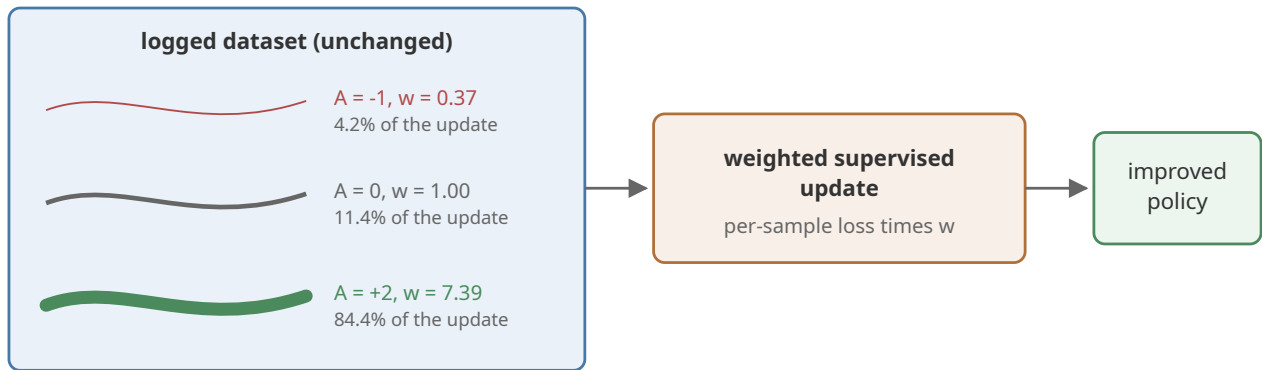


Figure 4.2 — AWR as a picture. Notice that the box on the left never changes: no episode is deleted and no new action is invented. Line thickness is the weight $w = e^{\hat{A}/\beta}$ at $\beta = 1$, and the percentages are each trajectory's share of the total weight. The failure still contributes a whisper of gradient, which is what keeps the policy anchored to data the robot can actually execute. One advantage per trajectory keeps the picture readable; in practice each frame carries its own $\hat{A}_t$.

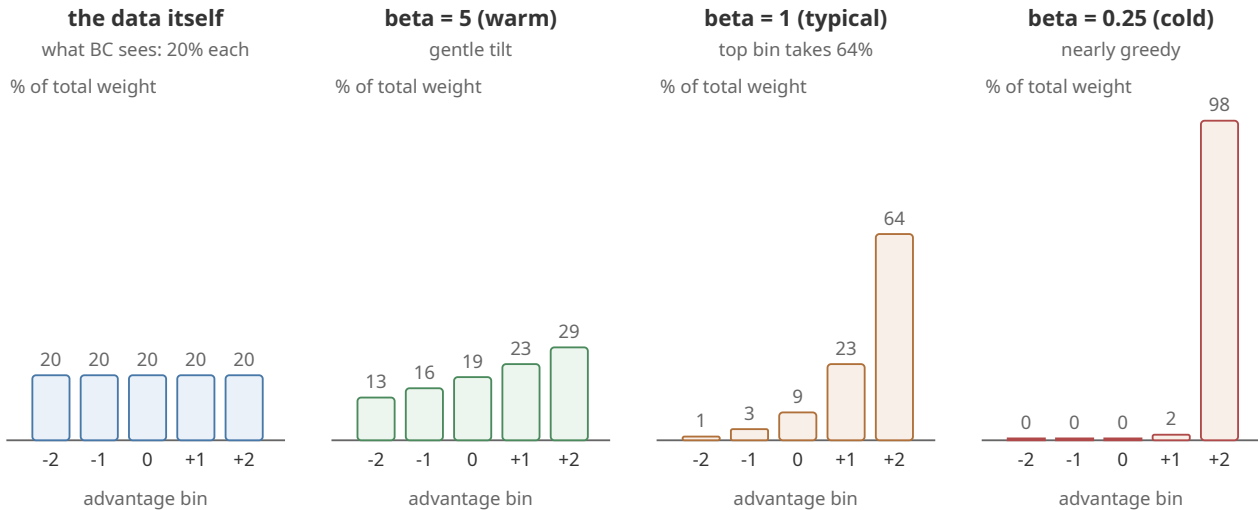


Figure 4.3 — The exponential tilt at three temperatures. The leftmost panel is a dataset with equal numbers of samples in five advantage bins; each later panel shows the same samples' share of the total weight after multiplying by $e^{\hat{A}/\beta}$, in percent. Notice that lowering β does not delete anything — it starves the lower bins. At $\beta = 0.25$ the top bin carries 98% of the update, so four fifths of your data has effectively stopped existing and you are one bad advantage estimate away from cloning noise.

4.4.5 Worked examples: making the temperature tangible

You have met the three numbers in this subsection already: $-1, 0, +2$ were the toy's advantages, and 0.37, 1.00, 7.39 their tilt factors at $\beta = 1$. What follows is the same arithmetic read from the training loop's side, with one change of scale: each advantage now belongs to a whole trajectory instead of one action at one state.

One clarification, because it decides what you build. The key equation weights per *sample* $i = (o_i, a_i)$ — that is, per frame. The examples below use one advantage per trajectory purely to keep the numbers checkable by hand. In a real implementation every frame carries its own $\hat{A}_t$ from GAE, and a trajectory-level weight is the special case where every frame in an episode is handed the episode's advantage. LeHome is per frame: frame i is drawn with probability proportional to $\exp(\text{clip}(\hat{A}_i, -2, 2))$.

Three logged trajectories with estimated advantages $\hat{A}_1 = -1, \hat{A}_2 = 0, \hat{A}_3 = +2$. Take $\beta = 1$ first, so the weight is just $e^{\hat{A}}$:

$$w_1 = e^{-1} \approx 0.37, \quad w_2 = e^0 = 1.00, \quad w_3 = e^2 \approx 7.39.$$

Now normalize, because what matters is each sample's *share* of the gradient, not its raw weight. Sum first, then divide, one line at a time:

```
sum    = 0.37 + 1.00 + 7.39 = 8.76
share1 = 0.37 / 8.76 = 0.042  -> 4.2%
share2 = 1.00 / 8.76 = 0.114  -> 11.4%
share3 = 7.39 / 8.76 = 0.844  -> 84.4%
check: 4.2 + 11.4 + 84.4 = 100.0%
```

Read that back into behavior: the update acts as if trajectory 3 had been duplicated about seven times, while trajectory 1 whispers at a third of its former volume. Nothing was deleted, and that residue is useful — it keeps the policy pinned to the data's support. These shares are the probabilities of π^* from 4.4.3 to the decimal, exactly as 4.4.4 promised.

Now the same three advantages with $\beta = 5$. Divide each advantage by 5 *before* exponentiating:

```

w1 = exp(-1/5) = exp(-0.2) = 0.819
w2 = exp( 0/5) = exp( 0.0) = 1.000
w3 = exp( 2/5) = exp( 0.4) = 1.492

sum      = 0.819 + 1.000 + 1.492 = 3.311
share1   = 0.819 / 3.311 = 0.247  -> 24.7%
share2   = 1.000 / 3.311 = 0.302  -> 30.2%
share3   = 1.492 / 3.311 = 0.451  -> 45.1%

```

Same data, same advantages, completely different training run. At $\beta = 1$ the best trajectory outweighs the worst by a factor of $7.39/0.37 \approx 20$; at $\beta = 5$ that ratio is $1.492/0.819 \approx 1.8$. The failure went from 4.2% of the update to nearly a quarter of it. In the language of 4.4.3, you tightened the leash from about 0.57 nats to about 0.03 by moving one knob. If your advantages are noisy — and on a real robot with a learned value function they are — the larger β is the conservative choice, at the price of learning more slowly.

Finally, watch the thing that will actually bite you in practice. Suppose a fourth trajectory got lucky and its estimated advantage came out at $\hat{A}_4 = +8$. At $\beta = 1$:

$$w_4 = e^8 \approx 2981, \quad \text{sum} = 0.37 + 1.00 + 7.39 + 2981 = 2989.76,$$

$$\text{share}_4 = \frac{2981}{2989.76} = 0.9971 \rightarrow 99.71\%, \quad \text{share}_3 = \frac{7.39}{2989.76} = 0.0025 \rightarrow 0.25\%.$$

Trajectory 3, which was carrying 84.4% of the update a moment ago, now carries a quarter of one percent. One sample — possibly one *lucky* sample — has become the entire training set. This is not a rare pathology; it is what exponentials do, and it is why the two defenses in the pitfall box are not optional.

Note the difference from the $+60^\circ$ action of 4.4.3: that one was never logged, so no advantage could give it weight. This trajectory *was* logged, so the tilt is entitled to amplify it, and only the scale you allow $\hat{A}$ stands between a noisy estimate and a collapsed run.

You met the same instinct in Chapter 3. PPO's ratio ρ_t is also unbounded, and Section 3.4 tamed it by clipping to $[1 - \epsilon, 1 + \epsilon]$. The clip at 20 in the code below is blunter: PPO's clip removes the incentive to move further (3.4.4 called it an incentive, not a constraint), while this one caps the weight outright.

```

# AWR: one training step, any supervised policy loss
batch = replay.sample()                # (o, a, A_hat) triples
A = (batch.A - A_mean) / (A_std + 1e-8) # normalize advantage scale
w = clip(exp(A / beta), max=20.0)       # tilt + clip
loss = (w * policy_loss(theta, batch.o, batch.a)).mean()
# policy_loss: -log pi_theta(a|o) for a Gaussian head, MSE for a
#               deterministic one, a flow-matching regression loss
#               for a generative one (Chapters 6 to 8)
theta = theta - lr * grad(loss)

```

ON YOUR RIG

Appendix B, rung E4, is this section on your own rl-rover logs: roll out a mediocre early checkpoint for about 500 episodes, then train one policy by plain regression onto its actions and one with $w_i = \exp(\hat{A}_i/\beta)$, $\hat{A}_i = G_i - V(s_i)$ from the checkpoint's own critic. Normalize $\hat{A}$ and clip w at 20. Then push β toward zero and watch the effective sample size collapse — dead data appearing on data you collected. Keep the weight histogram and be able to say what β does to its shape, as the rung asks. You cannot compute the leash radius of 4.4.3 here, because a continuous state holds one logged action rather than three hundred, but the histogram narrowing to a spike is the same leash lengthening, read from the weights instead.

PITFALLS

The exponential is unforgiving. Unnormalized advantages (scale 50 instead of 1) make every weight either 0 or an overflow — always standardize $\hat{A}$ per batch or per dataset before `exp`, and clip w , for example $w_i \leftarrow \min(w_i, 20)$. Too small a β creates *dead data*: the effective sample size collapses to a handful of trajectories, the policy overfits them, and it drifts off-support anyway. A cheap diagnostic is to log the effective sample size $(\sum_i w_i)^2 / \sum_i w_i^2$ each epoch. If it is a small fraction of your batch, or if most weights sit near zero, raise β : you are closer to greedy than you think. On the three-trajectory example above it is 1.38 at $\beta = 1$ and 2.81 at $\beta = 5$, out of three.

4.5 AWAC in one section

AWR as stated has a practical weakness: where does $\hat{A}_i$ come from? The plain answer is a Monte Carlo return minus a value baseline, which needs complete episodes and describes the *behavior* policy's performance from that state onward. That is noisy, it wastes partial trajectories, and it does not improve as your policy does.

AWAC (advantage-weighted actor-critic; Nair et al., 2020) keeps the weighted-regression policy update exactly as it is and changes only where the advantage comes from. It trains an action-value critic $Q(s, a)$ with off-policy temporal-difference backups over the whole buffer, and sets

$$\hat{A}(s, a) = Q(s, a) - \mathbb{E}_{a' \sim \pi_\theta} [Q(s, a')],$$

Name the pieces. a is the action actually recorded in the dataset at state s . a' is a *candidate* action — not read from the dataset, but drawn fresh from the current policy π_θ . Averaging $Q(s, a')$ over a handful of such draws is precisely an estimate of $V^{\pi_\theta}(s)$. So this is the same $A = Q - V$ you know from Chapter 3, with the baseline computed on the fly by sampling rather than by a second network. Note what it requires: the ability to *sample* from π_θ and to evaluate Q at those samples. It does not require the probability of any action. Keep that in mind for the next section.

If 4.2 stayed with you, that should worry you: querying a critic at actions the policy chose for itself is exactly the pattern that inflated the value surface and sent the wrist to $+60^\circ$. AWAC survives it because of what the actor update does. The actor is trained by weighted regression onto *logged* actions, which pins π_θ near π_β ; a policy pinned to the data's support draws its a' from inside that support, so the Q queries are ones the data can answer. But be clear about the status of that argument — the safety is conditional, not structural. Set β low enough, or fine-tune long enough with a stale critic, and AWAC drifts off support like anything else. Plain AWR cannot, because it never asks a critic anything.

Because both the critic and the actor update are fully off-policy, AWAC is built for the workflow real robots want: *pretrain offline* on demonstrations and logged data, then *fine-tune online*, still reusing everything in the replay buffer. Its selling point is the absence of the performance crash many methods suffer at the offline-to-online handoff: either their constraint is too rigid for the policy to move past the demonstrations once fresh data arrives, or their critic must effectively be relearned when the distribution shifts, and the policy degrades while it does.

ON YOUR RIG

Your HIL-SERL pipeline answers the same data-efficiency problem a different way: it is RLPD-family — off-policy Q-learning in the style of SAC (soft actor-critic: a Q-function plus an entropy-regularized Gaussian actor) over a replay buffer that mixes demonstrations, your pad interventions, and autonomous rollouts. That works because its small Gaussian action head has exact log-probabilities and cheap critic queries. AWR trades some of that per-sample efficiency for the one thing Q-learning-through-the-actor cannot offer: total indifference to the policy's architecture. Same problem, two off-policy answers — and Chapter 13 comes back to why the intervention data in that buffer deserves extra weight.

4.6 The property that matters

4.6.1 The shopping list

Now the hinge of this book. Look carefully at what the AWR update actually consumes:

- samples $(\mathbf{o}_i, \mathbf{a}_i)$ from a dataset;
- a supervised loss that scores how well the policy reproduces $\mathbf{a}_i$ from $\mathbf{o}_i$;
- one scalar weight w_i per sample.

That is the complete list. And here is what it never does: it never evaluates $\log \pi_\theta$ at an arbitrary action of its own choosing, and it never forms an importance ratio $\rho_t(\theta)$. Neither is a lucky accident — the whole point of the projection step in 4.4.4 was to turn every expectation into a sum over samples you already have.

The consequence is large. The $\log \pi_\theta(\mathbf{a}_i|\mathbf{o}_i)$ term in the key equation can be replaced by *whatever surrogate loss you already use to train this policy in a supervised way*: mean-squared error on actions for a deterministic head, a diffusion denoising loss (Chapter 6), a flow-matching velocity-regression loss (Chapter 7).

That substitution deserves better than a wave of the hand, because four chapters lean on it. For diffusion the case is solid: the denoising loss is a variational lower bound on the log-likelihood, so minimizing it is a principled stand-in for maximizing $\log \pi_\theta$, and weighting it by w_i weights a bound on the quantity you meant to weight. For flow matching the link is looser — the velocity-regression loss is not a likelihood bound in the same clean way, and the swap is a well-tested heuristic rather than a theorem. Be plain about the cost. Once you substitute, the weighted objective is no longer exactly the KL projection of π^* that 4.4.4 derived, and nobody can hand you a proof that it is. What is true, and what the empirical record supports, is that reweighting a regression loss toward high-advantage samples moves the fitted distribution the way you want. You are trading a guarantee for a policy class that can represent multimodal robot behavior at all. That trade is the premise of Part III, and it is worth making with your eyes open.

4.6.2 Why PPO cannot follow

PPO cannot make this move, and the reason is structural. Its objective is built from

$$\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta_{old}}(\mathbf{a}_t|\mathbf{s}_t)},$$

which requires exact action likelihoods under two different networks. For a Gaussian head that is one line of code. Chapters 5 through 8 introduce policies for which it is not available at all. Diffusion and flow-matching policies produce an action by running an iterative sampling procedure, and expose no tractable $\pi_\theta(\mathbf{a}|\mathbf{o})$ at the end of it. Vision-language-action models (VLAs: a pretrained vision-language backbone with an action head bolted on, the subject of Chapter 8) are built on those samplers, so they inherit the problem. Action chunking is a separate matter — emitting $\mathbf{a}_{t:t+H}$ in one shot is not an iterative procedure and is not by itself what destroys the density. It compounds the difficulty from the other side: the object whose probability you would need is then a thirty-step trajectory rather than a single command. You can sample from any of these policies all day. You cannot ask them how likely a particular action was. PPO's central quantity is simply unavailable.

Those same policies are, however, *trained* by supervised regression. That is the whole reason they exist and scale. So here is the rule worth memorizing:

Any policy class you can train with supervised learning, you can improve with AWR. Weight the per-sample loss and you are done.

This is why weighted behavior cloning, not PPO, is what scales up to VLAs, and why the rest of this book looks the way it does. Chapter 9 lays the two worlds side by side; Chapter 10 shows how RECAP turns the advantage from a weight into a *conditioning input*, a further step down the same road. Figure 4.4 is the decision table.

	exact log-prob available	no tractable log-prob
data from this policy version only	PPO / TRPO — Chapter 3 Gaussian head, ratio is computable. Your rl-rover in simulation. Cheap samples assumed.	AWR anyway — Chapter 4 Fresh data does not restore a ratio. Score your own rollouts, reweight. PPO is unavailable even here.
data from a mixed pool of past versions	AWAC, RLPD / SAC — Ch. 4 Critic over the whole buffer. Your hil-serl SO-101 rig. Offline pretrain, online finetune.	AWR, then conditioning — Ch. 10 Weight the denoising or flow loss. Diffusion policies, VLAs; Ch. 6 to 8. RECAP; the LeHome flywheel.

the right-hand column is the whole of Part III

Figure 4.4 — Which family do you need? Two questions decide it: was this data collected by the policy version you are updating, or drawn from a pool of past versions; and can you evaluate the probability of a given action under your policy. Notice that the second question, not the first, is what separates PPO from everything else in this book — the top-right cell exists precisely because collecting fresh rollouts does not hand a flow-matching policy a usable importance ratio. LeHome sits bottom right: its pool holds roughly 12,500 rollout episodes from many checkpoints, decayed and resampled, which is off-policy by the definition in 4.1.

4.6.3 The same weight, applied at the sampler

One implementation note that seems trivial and is not. Multiplying the per-sample loss by w_i is the same in expectation as leaving the loss unweighted and instead *drawing* sample i with probability proportional to w_i . Both put the same emphasis on the same frames — but only up to one overall constant, the mean weight $\frac{1}{N} \sum_i w_i$, because loss weighting averages $w_i L_i$ over N uniform draws while sampling averages L_i over draws from $p_i = w_i / \sum_j w_j$. That constant rescales the gradient and is absorbed by the learning rate. Worth knowing on the day you switch an existing run from one form to the other and your effective step size quietly changes.

On a small MLP the difference does not matter. On a VLA it matters enormously: a frame you never draw is a frame you never decode, never move to the GPU, and never run a vision encoder on. With loss weighting you pay full price for a sample whose weight is 0.01; with sampling you skip it. That is the version Larchenko used.

```
# same update, weight moved into the sampler
p = softmax(clip(A_hat, -2.0, 2.0) / beta) # sampling law over frames
idx = multinomial(p, num_samples=batch_size) # low-weight frames never load
loss = policy_loss(theta, o[idx], a[idx]).mean() # plain, unweighted
# auxiliary heads (extra prediction outputs on the same backbone,
# Chapter 8) want the *original* data distribution, not the tilted one,
# so debias them with 1 / (N * p_i); the full form also divides by
# the episode length
```

IN THE LEHOME SOLUTION

AWR is one of the two pillars of the garment-folding solution (Larchenko, 2026); the other is RECAP (Chapter 10). The policy is derived from $\pi_{0.5}$ (a flow-matching VLA from Physical Intelligence, 2025a, dissected in Chapter 8), so it has no usable log-probabilities and improvement is exactly Section 4.6. He applies the weight at the sampler: frame i is drawn with probability proportional to $e^{\text{clip}(\hat{A}_i, -2, 2)}$, and the flow-matching loss itself is plain unweighted MSE. Failed episodes have their accumulated checkpoint rewards withdrawn (Chapter 11), pushing their advantages, and so their sampling probability, down. Advantages come from the policy's own success-probability head with a damped baseline, also Chapter 11.

CHECKPOINT

- Price out an on-policy run on real hardware in hours of human supervision, and explain why the data is discarded after the update.
- Explain, with a concrete action the dataset never contains, why naive offline Q-learning inflates its own value estimates — and why bootstrapping spreads the error.
- State behavior cloning's objective and its exact ceiling, and give an example of mixed-quality data where that ceiling bites.
- Derive AWR's shape from the KL-constrained problem: the closed-form exponential tilt, then the forward-KL projection onto the network as weighted maximum likelihood — including where the entropy term and $Z(\mathbf{s})$ went, and why the tilt can never reach an action the data does not contain, which is the source of both AWR's offline safety and its inability to explore — and why the tilt is an exponential at all: what a squared-distance leash would have produced on the toy instead.
- Compute weights and normalized shares from a list of advantages at two different temperatures, and say what $\beta \rightarrow 0$ and $\beta \rightarrow \infty$ each mean, and read a temperature as a price: name the leash radius $\beta = 1$ and $\beta = 5$ each buy on the toy of 4.4.2.
- State the rule: any policy class you can train with supervised learning, you can improve with AWR — explain why PPO's importance ratio blocks the same move, and say honestly what swapping a denoising or flow-matching loss in for $\log \pi_\theta$ costs in rigor.

Where to go deeper. The AWR paper (Peng et al., 2019) is short and readable, and its appendix contains, in compressed form, the derivation 4.4.3 unpacked at length; its ancestor is reward-weighted regression (Peters & Schaal, 2007). For the offline-pretrain-then-online-fine-tune story, read AWAC (Nair et al., 2020). Appendix A places both papers in the reading order for this book. RLPD (Ball et al., 2023) — the off-policy Q-learning alternative already running on your rig — and the conservatism-based offline methods this chapter deliberately routed around are optional context beyond the guide.